



University of
Southern Denmark

EMBEDDED SYSTEMS

DRONE MASTER

1. SEMESTER

GROUP 8

Embedded Mini Project

made by:

Adam Kolodziej adkol23@student.sdu.dk

May 31, 2026

Contents

1	Introduction	3
1.1	Industry needs	3
1.2	Overview and mechanical parts	3
2	Github	4
3	AI	4
3.1	Dataset	4
3.2	Training	5
4	HLS	6
4.1	Testing	6
5	Vivado	6
5.1	DMA IP	7
5.2	Utilization	8
6	Vitis	8
7	User-space	9
7.1	"usb_cam"	9
7.2	"AI_node"	9
7.3	"nut_position_orientation"	9
7.4	"motor_controller"	10
7.5	"system_movement"	10
7.6	DMA	10
8	Computer Vision	10
8.1	Homography	11
8.2	Arm movement	11
8.2.1	Calculating tool rotation and exact position of the nut - "nut_position_orientation"	11
8.2.2	Test node - "test_publish_image"	13
9	Conclusion	13

List of Figures

1	OnRobot ‘intelligent screwdriver’	3
2	Device	4
4	AI training results	5
3	Dataset	5
5	Testing results	6
6	Vivado block diagram	7
7	DMA IP	7
8	FPGA utilization	8
9	Bare metal result	8
10	ROS2 system diagram	9
11	Hex nut	11
12	Image after process stages (a) original image (b) after masking	12
13	Image after process stages (a) dilatation (b) eROS2ion (c) bit-wise inverted	12
14	Original image with computed data	13
15	Output data (a) custom messages (b) logger output	13

1 Introduction

1.1 Industry needs

The following Mini Project deals with the subject related to industry. Companies want to optimize processes to make the as fast and as economic as possible. It is especially true for production lines where the most important is effectiveness. This factor pushes factories to become more and more automated and autonomous.

To achieve that, many processes are done by robots. One of this process is screwing the bolts and nuts. We were supposed to create control system for that kind of machine.



Figure 1: OnRobot 'intelligent screwdriver'

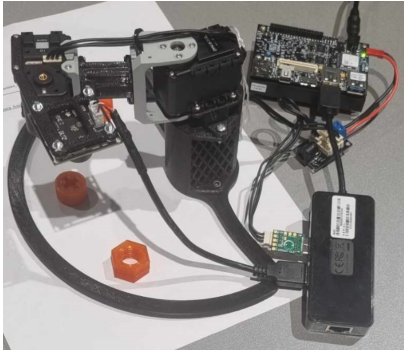
1.2 Overview and mechanical parts

System uses robotic arm built with designed by us 3D printed parts, two Dynamixel AX-12A servomotors and the camera. Everything is connected to the Ultra96-V2 board which is the brain of the whole system. Workflow which should be achieved can be split to 4 stages:

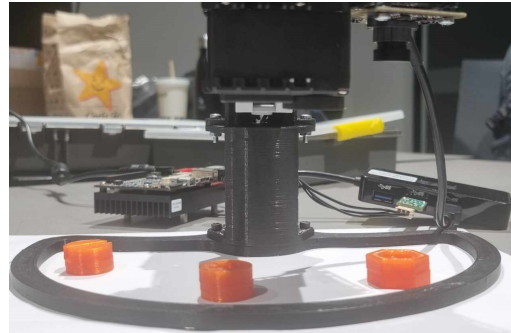
- Area scan using servomotor and camera
- AI based nut or bolt detection
- Computer Vision based calculation of tool orientation and nut position
- Arm and tool alignment to be able to potentially screw the bolt (ROS2)

Bottom part of the 3D stand consists of a bracket which guarantees stability but what turned out needs to be removed from camera feed. To the bracket, tower was attached to be able to adjust how high the camera is (it wasn't used but it is always nice to have more options). Next we printed arm motor mount. To attach arm and the tool motor plus the camera we used additional parts from the

kit and another 3D prints. It is important that the camera's lens are mounted on the same radius like tool axis.



(a)



(b)

Figure 2: Device

2 Github

GitHub is divided to a few repositories because it was better to work on AI, scripts, ROS2 nodes etc. on separated repos.

- main used for the MiniProject: https://github.com/Lpopvil/MiniProject_ES
- nodes and scripts used for task connected with computer vision: https://github.com/u000000/ES_images
- motor operations: <https://github.com/lucasrj/embedded-systems-ros-motor>
- custom ros2bag tools: https://github.com/u000000/ros2bag_tools

3 AI

3.1 Dataset

Provided GitHub consists of neural network which was trained to read numbers from 0 to 9. For Mini Project purposes it had to be modified and taught to detect 3/4 different nuts or bolts. So we designed and printed 3 type of nuts.

Dataset was collected by recording the video with every screw. During that process we were recording the camera topic running so we had to somehow extract images from ros2bag. We wrote custom ros2bag tool "db3tool" which also can be found on GitHub. Basically node has a subscriber which will

```

Epoch 47/50
3/3 [=====] - 0s 6ms/step - loss: 0.4320 - accuracy: 0.8830 - val_loss: 0.4636 - val_accuracy: 0.8644
Epoch 48/50
3/3 [=====] - 0s 6ms/step - loss: 0.4238 - accuracy: 0.8839 - val_loss: 0.4543 - val_accuracy: 0.8703
Epoch 49/50
3/3 [=====] - 0s 8ms/step - loss: 0.4161 - accuracy: 0.8865 - val_loss: 0.4473 - val_accuracy: 0.8743
Epoch 50/50
3/3 [=====] - 0s 8ms/step - loss: 0.4086 - accuracy: 0.8880 - val_loss: 0.4394 - val_accuracy: 0.8762
Model: "sequential"
-----
Layer (type)                Output Shape                Param #
-----
flatten (Flatten)           (None, 100)                  0
-----
dense (Dense)                (None, 32)                  3200
-----
dense_1 (Dense)              (None, 16)                   512
-----
dense_2 (Dense)              (None, 10)                   160
-----
Total params: 3,872
Trainable params: 3,872
Non-trainable params: 0
-----
Inference time for 1140 test image: 0.03534746170843945 seconds
Test loss, test acc: [0.5822751522064209, 0.7921052575111389]

Test Image 1 - True Label: 1
Predicted Label: 1

```

Figure 4: AI training results

take every frame from the video and save it as a .png file.

Afterwards, pictures were cropped to 400x350 pixels not to have stand bracket in the view. It was done using the "Image.copy()". Next images were augmented to get the best training results.

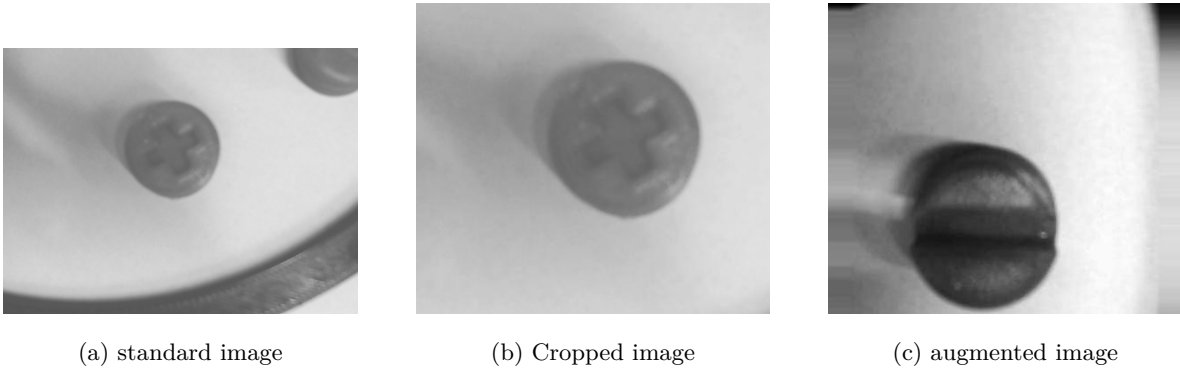


Figure 3: Dataset

3.2 Training

To get maximum accuracy training was done with images which were also rotated. Used AI has 3 layers: first two utilize "ReLU" method and the third one the "softmax" method. Adam optimizer was used to reduce the loss function. The training was done with batch size of 2000, validation split of 10% and epochs of 50. We achieved recognition accuracy between 72% and 79%. After training weights were saved to be used during next steps.

4 HLS

Having AI working, we had to implement it on FPGA on the Ultra96V2. To do that we created IP which top function was called "nn_inference". It has two ap_axis streams transferring signed 32bit data to communicate via DMA. One of them is an input to receive 10x10 flattened images and the second one is an output to send results. Pragma HLS was used to be able to set ports used later in Vivado. Basically, when image is received, it's normalized, converted to ap_fixed and used by layers of AI. Then, the highest possibility is out. We found out that it's better to use ap_fixed instead of float because then we can reduce number of used FPGA by about half.

4.1 Testing

Test bench was written in HLS. As a testing data we used 3 images which were transferred using inputs stream as 1x100 array and processed by IP. We printed the results to see if everything works correctly. The system worked properly.

```
NN Prediction: 0

NN Prediction: 1

NN Prediction: 0

Success
WARNING: Hls::stream 'hls::stream<hls::axis<ap_int<32>, 0ul, 0ul, 0ul>, 0>.2' contains leftover data, which may result in RTL simulation hanging.
The maximum depth reached by any of the 2 hls::stream() instances in the design is 100
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] ***** CSIM finish *****
INFO: [HLS 200-111] Finished Command csim_design CPU user time: 18.87 seconds. CPU system time: 2.62 seconds. Elapsed time: 18.54 seconds; current allocated memory: 191.622 MB.
Finished C simulation.
```

Figure 5: Testing results

5 Vivado

We decided to use similar solution to one we used in Assignment 2 - Direct Memory Access. Like before we used AXI Direct Memory Access and AXI_SmartConnect and Processor System Reset blocks. We also activated high performance ports of Zynq UltraScale+ MPSoC to be able to communicate with "nn_inference" block.

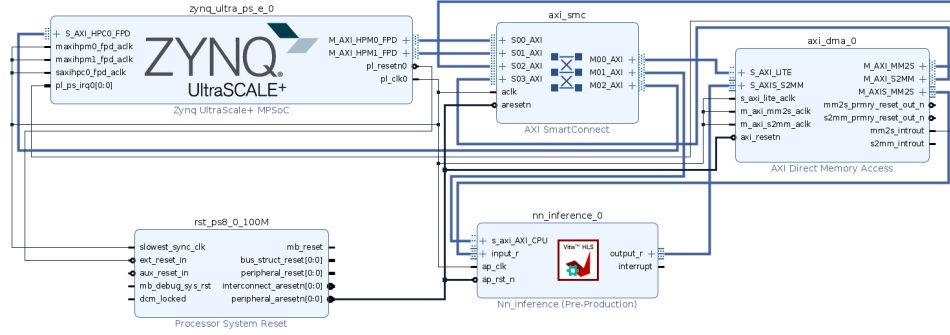


Figure 6: Vivado block diagram

5.1 DMA IP

Address Width is set to 64bit because used Ubuntu is 64bit operating system. Both, Read and Write channels were enabled and set memory map and stream data width to 32bit because streamed images are 32bit. M_AXIS_MM2S and S_AXIS_S2MM are used to stream the images to AI block and stream back the results to system memory.

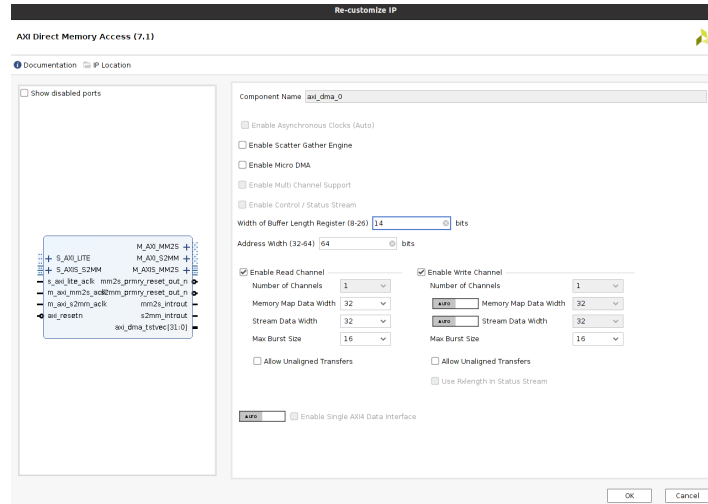


Figure 7: DMA IP

5.2 Utilization

When float were used - almost all FPGA were used but in figure 8b we can see that when we used `ap_fixed`, system used about 1/3 of LUT. It means that we can potentially run more advanced AI or run computations at the same time.

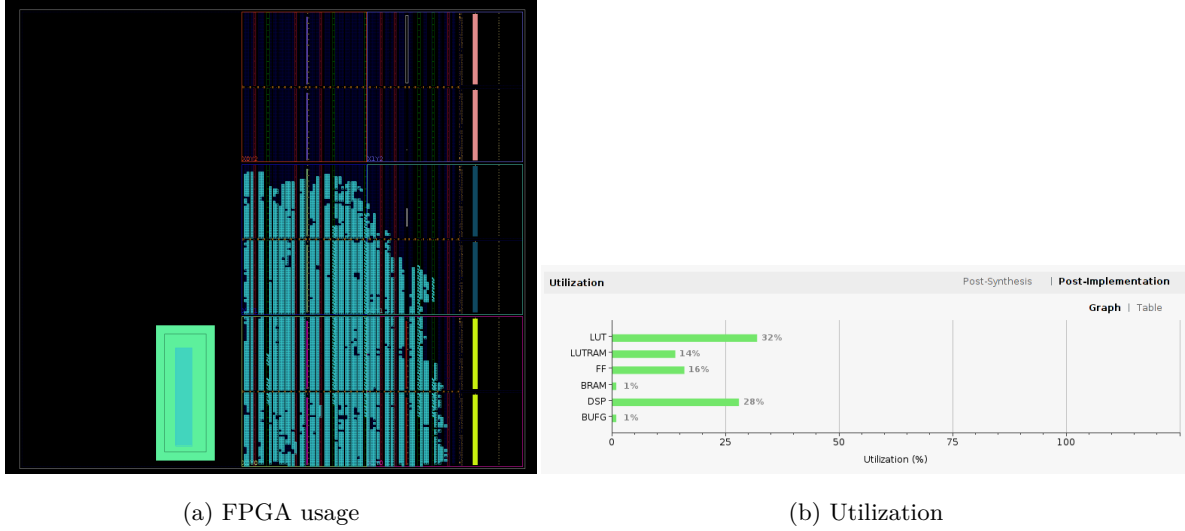


Figure 8: FPGA utilization

6 Vitis

In Vitis bare metal test was created. We wanted to test "nn_inference". At the beginning DMA IP, AI IP and UART port are initialized. We use Python script to prepare data for AI. 10x10 greyscale image is sent via UART to simulate data coming from camera. Then image is converted to 32 array and sent to DMA. AI computation is done and we receive results. Unfortunately we always get output: 2 which is not correct. We can't really tell what is wrong. Maybe we have some problems when we transfer data via UART.

```
Label: 0 Filename: num_2184_cropped_augmented_resized1.jpg Serialport: /dev/ttyUSB1
entering while loop
Network output: 2
```

Figure 9: Bare metal result

7 User-space

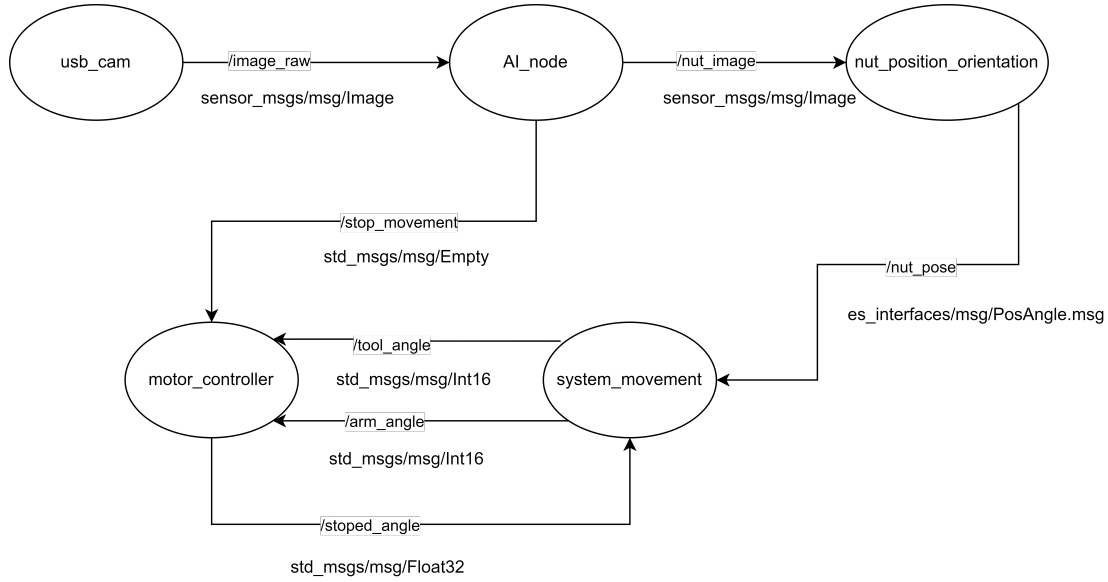


Figure 10: ROS2 system diagram

7.1 "usb_cam"

First used node was "usb_cam" which is standard node in ROS2. It publishes video feed from connected USB camera.

7.2 "AI_node"

This node gets an image from "usb_cam" node, crops it to 400x350, scales to 10x10 and converts it to grayscale. It communicates with FPGA using AXI Interconnect. It communicates with DMAs and "nn_inference". It uses the memory access file from the reserved memLKM kernel modules so it can write data to reserved memory. Edited image is transferred to reserved memory space. Information about its address and size is sent to DMA IP and AI computation starts. If the hex nut is found, then original image is sent to "nut_position_orientation" node and stop message is sent to "moto_control".

7.3 "nut_position_orientation"

This node was detailedly described in the section about Computer Vision 8.

7.4 "motor_controller"

It is based on the dynamixel_sdk provided by manufacturer of the servos. Every servo has ID. Servos are connected in series so they are on the same port. Two subscribers are used to control servos and the third one is a stop command. Arm servo holds almost whole device so it's set to move slowly so scan done by camera is clean enough to use. We don't have any restrictions about tool servo so it's set to move with high speed. When controller gets stop command, it saves motor position and sends it to "system_movement" node.

7.5 "system_movement"

This is final node which controls movement of the robotic arm when nut is found and everything is calculated. First it calculates the nut position corresponding to the home point. Then it will calculate how much the arm needs to move to put tool above the nut. Finally it will send this information with tool movement degrees (from "nut_position_orientation") to "motor_controller" which will control servos.

7.6 DMA

To be able to use DMA with ROS2 we wrote AXI DMA driver which is similar to the one used in Vitis. It can access the AXI Layer. To do that it does memory mapping of the UIO file for the DMA IP (32bit unsigned integer pointer). The pointer is used to write and read data to AXI Layer. It is written as a class. When every data is initialized, SimpleTransfer is used to output the stream of data or save stream data into memory.

8 Computer Vision

All packages with nodes and test nodes are stored in this GitHub repository:

https://github.com/u000000/ES_images

"nut_position_orientation" package consists of two nodes operating on camera frames and one script used to do homography transformation.

"es_interfaces" package consists of custom messages created to easier operate on data which needs to be transferred between main nodes.

8.1 Homography

Before taking care of calculating data, which needs to be done to move robotic arm correctly, it's necessary to perform camera feed calibration. We had to calculate transformation matrix which is capable of converting a set of homogeneous coordinates from one plane to another. It is very useful to be able to work only on one set of coordinates. We chose to have home point in front of the tower (close to the axis of arm motor movement). We used OpenCV function called `getPerspectiveTransform()`. As an output we get the matrix which will change original coordinates to millimeters accordingly to new home point.

8.2 Arm movement

AI part is responsible for finding determined nuts or bolts. The nut which is supposed to be screw in is a hex one (11). When hex nut is detected by AI, the frame and position of the arm is saved by running ROS2 node. Then the frame is sent to ROS2 node called "nut_position_orientation".

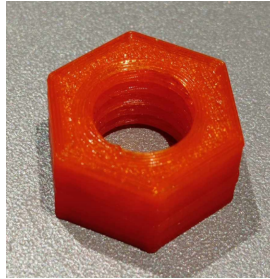
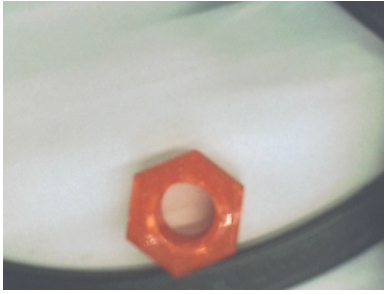


Figure 11: Hex nut

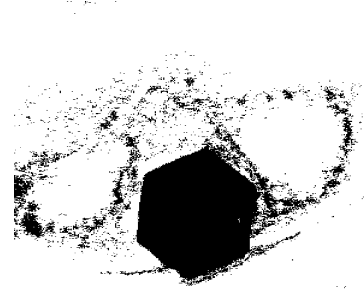
8.2.1 Calculating tool rotation and exact position of the nut - "nut_position_orientation"

This ROS2 node gets as an input "Image" message which is a format used by camera node. Subscriber will take only "Image" format. To operate on that data, it is converted to cv2 format using "`cv_bridge.CvBridge()`". To be able to find exact position of the nut and its orientation, image have to be prepared. First it is converted from BGR to HSV color space which gives better results when you want find contours etc. After that masking is performed using OpenCV "`inRange()`" with connection to determined lower and upper bounds. 12

Subsequently, dilatation and eROS2ion are performed. Dilatation consists of convolving an image with some kernel - in our case we used a square 15x15. EROS2ion is really similar to dilatation. It computes a local minimum over the area of given kernel - in our case we used a square 10x10. Afterwards, bit-wise inversion is done and an image is ready to use. 13



(a)



(b)

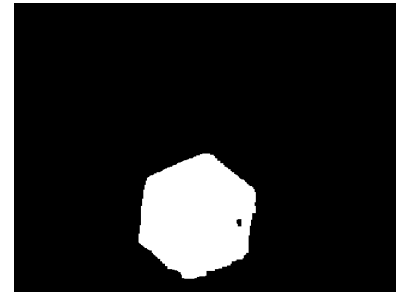
Figure 12: Image after process stages (a) original image (b) after masking



(a)



(b)



(c)

Figure 13: Image after process stages (a) dilatation (b) eROS2ion (c) bit-wise inverted

First, we look for contours and their center points. Code can operate on multiple contours but in our case prepared image should always have only one - hex screw and nothing else. Center point is contour's center of mass. Then nut position variable is created and filled. This variable was created by us to have easy access to center point when message is sent to next node.

Next stage is calculating tool movement angle. Code is quite long so I won't paste it here - everything is on GitHub. One of tasks was to align the tool with the detected nut. To achieve that we look for the furthest point of the contour from its center of mass. That way we can find of the corners of hex nut. Using that information we can calculate the angle. It is divided to 4 quarters to always find the most optimal solution. Value is computed in radians but to simplify things we decided to convert it to degrees. We also had to keep in mind that camera mounting point is a bit rotated so we had to add an offset.

Figure 14 shows original image with computed data. Blue dot indicates the furthest point of the contour from its center of mass. Brown dot in the middle is the center of mass of the contour and the line indicates direction to robotic arm rotation axis.

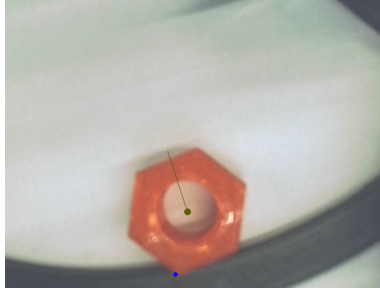


Figure 14: Original image with computed data

As I mentioned before to make everything as easy to work with as possible we created custom messages which are supposed to be used to transfer data between ROS2 nodes. These are "Pos" and "PosAngle". "Pos" is the part of "PosAngle". "Pos" consists of data where exactly is the center of the nut correspondingly to center of the camera feed. "Angle" is a tool movement angle. (Figure 15) To sum up, if we publish data using "PosAngle" format we can easily access "Pos" part as a point with "x" and "y" coordinates as "position" and "angle" directly.

```

adam@08080Thinkbook: ~ $ ros2 interface show es_interfaces/msg/Pos
float64 x
float64 y
adam@08080Thinkbook: ~ $ ros2 interface show es_interfaces/msg/PosAngle
es_interfaces/Pos position
float64 x
float64 y
float64 angle

```

(a)

```

[INFO] [1705079610.884517890] [nut_position_orientation]: Looking for images from AI
[INFO] [1705079620.145767372] [nut_position_orientation]: Nut position: es_interfaces.msg.Pos(x=-17.0
, y=113.0)
[INFO] [1705079620.147607133] [nut_position_orientation]: Angle: -29.305527054658730

```

(b)

Figure 15: Output data (a) custom messages (b) logger output

8.2.2 Test node - "test_publish_image"

To test "nut_position_orientation" node we wrote "test_publish_image" node which publishes a frame in "Image" format. During testing, "nut_position_orientation" subscribe "test_publish" so the situation is basically identical as if it is data coming from camera node during true process. This node is also available on GitHub.

9 Conclusion

We did our best but unfortunately we didn't manage to complete the entire assignment. We had almost every part of the system tested and ready to use. We wanted to wrap everything together and see how it works but due to problems with implementing the FPGA system what was described in section 6. Nevertheless huge amount of work was done and many processes are proved to work. It is also possible to optimize AI and some other processes or make it more advanced.